Serverless Compute Standards

> **Document Owner:** Platform Engineering | **Status:** Approved | **Last Updated:** February 2026

Overview

Serverless compute is the default for all new workloads. This document defines when and how to use serverless across SQL Warehouses, Jobs, and DLT Pipelines.

Golden Rules

ID	Rule	Severity	WAF Pillar
SC-01	Serverless is default for all new workloads	Critical	Cost
SC-02	Serverless SQL Warehouses for all SQL	Critical	Cost
SC-03	Serverless Jobs for notebooks and Python	Critical	Cost
SC-04	Serverless DLT for all pipelines	Required	Cost
SC-05	Use latest serverless environment version	Required	Performance
SC-06	Pin all custom dependencies with explicit versions	Critical	Reliability
SC-07	Use workspace base environments for shared dependencies	Required	OpEx
SC-08	Never install PySpark on serverless compute	Critical	Reliability
SC-09	Configure private package repositories for internal libraries	Required	Security

---

Why Serverless First?

Benefit	Description
Instant startup	Seconds vs 5-10 minutes
Auto-scaling	No manual configuration
Pay-per-use	No idle time costs
Built-in Photon	2-8x performance
30-50% cost reduction	Compared to classic clusters

Classic Compute Exceptions

Exception	Justification	Approval Required
GPU workloads	ML training with GPUs	Platform Architect
24/7 Streaming	Long-running streams	Platform Architect

Exception	Justification	Approval Required
Specialty libraries	Native C/C++ dependencies	Platform Engineer

---

Serverless SQL Warehouses

Configuration

```yaml
resources:
  warehouses:
    analytics_warehouse:
      name: "[${bundle.target}] Analytics Warehouse"
      warehouse_type: "SERVERLESS"
      cluster_size: "Small"
      auto_stop_mins: 10
      enable_photon: true
      min_num_clusters: 1
      max_num_clusters: 4
```

Sizing Guide

Size	vCPUs	Use Case
2X-Small	4	Development
Small	16	Standard analytics
Medium	32	Heavy dashboards, Genie
Large	64	Complex queries

Serverless Jobs

Mandatory Configuration

```yaml
resources:
  jobs:
    my_job:
      name: "[${bundle.target}] My Job"

      # MANDATORY: Serverless environment
      environments:
        - environment_key: "default"
          spec:
            environment_version: "4"
            dependencies:
              - "pandas==2.0.3"

      tasks:
```

```
        - task_key: main_task
          environment_key: default
          notebook_task:
            notebook_path: ../src/my_notebook.py
```

Never Do This

```
# WRONG: Classic cluster configuration
job_clusters:
  - job_cluster_key: "my_cluster"
    new_cluster:
      spark_version: "14.3.x-scala2.12"
      node_type_id: "i3.xlarge"
```

Serverless DLT Pipelines

```
resources:
  pipelines:
    silver_pipeline:
      name: "[${bundle.target}] Silver Pipeline"
      catalog: ${var.catalog}
      schema: ${var.silver_schema}
      serverless: true        # MANDATORY
      photon: true            # MANDATORY
      channel: CURRENT
      edition: ADVANCED
```

SC-05: Use Latest Serverless Environment Version

Rule

All serverless notebooks and jobs must use the latest available environment version unless a specific version is pinned for stability during a release cycle.

Why It Matters

- Latest versions include security patches, performance improvements, and bug fixes
- Newer environment versions include updated pre-installed libraries
- Databricks recommends the latest version for the most up-to-date features

Implementation

Notebooks:

1. Open the **Environment** side panel

2. Under **Base environment**, select the latest **Standard** version (or **AI** for GPU workloads)
3. Click **Apply**

Jobs (Asset Bundles):

```
resources:
  jobs:
    my_job:
      environments:
        - environment_key: "default"
          spec:
            environment_version: "4"   # Always use latest stable version
            dependencies:
              - "pandas==2.0.3"
```

Anti-Pattern

```
# WRONG: Using an outdated environment version
environments:
  - environment_key: "default"
    spec:
      environment_version: "1"    # Outdated — missing security patches
and features
```

SC-06: Pin All Custom Dependencies with Explicit Versions

Rule

All custom dependencies in serverless environments must be pinned to explicit versions. No unpinned or range-based specifiers for production workloads.

Why It Matters

- Unpinned dependencies can change between runs, causing silent failures
- Reproducibility is impossible without version locks
- A library upgrade can break production pipelines without warning

Implementation

Notebook Environment Panel: Add dependencies using `requirements.txt` format with pinned versions:

```
pandas==2.0.3
pyarrow==14.0.2
scikit-learn==1.3.2
mlflow==2.17.0
```

Jobs (Asset Bundles):

```yaml
environments:
  - environment_key: "default"
    spec:
      environment_version: "4"
      dependencies:
        - "pandas==2.0.3"
        - "pyarrow==14.0.2"
        - "scikit-learn==1.3.2"
```

Custom Environment YAML (exportable and reusable): Export from notebook Environment panel via kebab menu > **Export environment**, then store in Workspace Files or UC Volumes.

Anti-Patterns

```yaml
# WRONG: Unpinned dependencies
dependencies:
  - "pandas"              # Any version — breaks reproducibility
  - "scikit-learn>=1.0"   # Range specifier — could pull breaking changes
  - "mlflow"              # Will upgrade silently between runs
```

SC-07: Use Workspace Base Environments for Shared Dependencies

Rule

Teams must use workspace base environments (admin-configured) or custom base environments (YAML files) to standardize dependencies across notebooks and jobs. Avoid ad-hoc per-notebook dependency installation.

Why It Matters

- Ensures all team members use the same library versions
- Eliminates "works on my notebook" deployment failures
- Reduces environment setup time for new team members
- Single source of truth for dependency management

Implementation

Create a custom base environment:

1. Configure a serverless notebook with required dependencies
2. Click kebab menu > **Export environment** to save as YAML
3. Store the YAML in a shared Workspace path: `/Workspace/Shared/environments/`

Use the shared environment:

- **Notebooks:** Select from **Base environment** > **Workspace environments**
- **Jobs:** Reference the YAML in the environment spec

Common utilities pattern:

```
/Workspace/Shared/common_utils/
├── helpers/
│   └── __init__.py        # Shared functions
├── pyproject.toml
```

Add as dependency: `/Workspace/Shared/common_utils`

Anti-Pattern

- Each developer installing different library versions ad-hoc in their notebooks
- Copy-pasting dependency lists across notebooks instead of using a shared base environment

---

SC-08: Never Install PySpark on Serverless Compute

Rule

Never install PySpark or any library that installs PySpark as a transitive dependency on serverless notebooks or jobs. This will terminate the session and cause errors.

Why It Matters

- Serverless compute provides a managed Spark runtime — installing PySpark conflicts with it
- The session terminates immediately with an unrecoverable error
- Libraries like `delta-spark`, `databricks-connect` (standalone), or custom packages that declare PySpark as a dependency will trigger this failure

Known Problematic Packages

Package	Why	Alternative
`pyspark`	Directly installs PySpark	Already available on serverless
`delta-spark`	Depends on PySpark	Use built-in Delta Lake support
`databricks-connect` (standalone)	Installs PySpark	Use the serverless-native version

Recovery

If PySpark is accidentally installed:

1. Remove the offending package from the **Dependencies** section
2. Click the arrow next to **Apply** > **Reset to defaults**

3. Restart the notebook session

SC-09: Configure Private Package Repositories for Internal Libraries

Rule

Workspaces that use internal Python packages must configure private package repositories as default pip sources. Do not hardcode repository URLs in individual notebooks.

Why It Matters

- Prevents exposure of internal package repository credentials in notebook code
- Standardizes package sourcing across all serverless compute
- Eliminates the need for users to manually specify `--index-url` or `--extra-index-url`
- Supports security scanning of internal packages

Implementation

Admin configuration:

1. Navigate to **Admin Settings** > **Workspace Settings** > **Default Python Packages**
2. Configure the private repository URL and credentials
3. All serverless notebooks and jobs will use this repository by default

Storing internal wheels:

```
/Volumes/<catalog>/<schema>/python_packages/
├── internal_utils-1.2.0-py3-none-any.whl
├── data_validators-0.5.1-py3-none-any.whl
```

Reference in dependencies:

```
/Volumes/engineering/shared/python_packages/internal_utils-1.2.0-py3-none-
any.whl
```

Anti-Pattern

```
# WRONG: Hardcoded repository URL in notebook
%pip install --index-url
https://user:token@internal.pypi.example.com/simple/ internal-utils
```

Cost Monitoring

```sql
SELECT
    usage_date,
    workspace_id,
    sku_name,
    SUM(usage_quantity) as total_dbus,
    SUM(list_cost) as total_cost
FROM system.billing.usage
WHERE sku_name LIKE '%SERVERLESS%'
    AND usage_date >= CURRENT_DATE - 30
GROUP BY 1, 2, 3
ORDER BY total_cost DESC;
```

Validation Checklist

SQL Warehouses

- [ ] Warehouse type is SERVERLESS
- [ ] Photon enabled
- [ ] Auto-stop configured (5-15 min)

Jobs

- [ ] `environments` block defined
- [ ] `environment_key` on every task
- [ ] No `job_clusters` or `new_cluster`

DLT Pipelines

- [ ] `serverless: true` set
- [ ] `photon: true` set
- [ ] No cluster configuration

Environment Management

- [ ] Latest environment version used (SC-05)
- [ ] All dependencies pinned with explicit versions (SC-06)
- [ ] Workspace/custom base environment used for team consistency (SC-07)
- [ ] No PySpark or PySpark-dependent libraries installed (SC-08)
- [ ] Private package repositories configured at workspace level (SC-09)

Related Documents

- Platform Overview
- Cluster Policies

References

- Performance Efficiency Best Practices
- Serverless SQL Warehouses
- Serverless Jobs
- Configure Serverless Environment
- Serverless Environment Versions
- Default Python Package Repositories